



Example: Disaggregated prefill

With LMCache as a KV cache transfer library, we can run disaggregated prefill with vLLM. Right now, LMCache uses NIXL as a transport layer to enable fast KV cache transfer via NVLink, RDMA, or TCP.

This guide demonstrates how to run LMCache with disaggregated prefill using a single prefiller and decoder setup (1P1D) on a single machine. The architecture splits the LLM inference into two stages: prefill and decode, running on separate GPUs for better resource utilization.

Prerequisites

Before you begin, ensure you have:

- At least 2 GPUs
- Python packages installed:
 - `lmcache` (0.2.1 or above)
 - `nixl` (Install instructions [here](#))
 - `vllm` (latest main branch)
 - `httpx`, `fastapi`, and `uvicorn`
- A valid Hugging Face token (`HF_TOKEN`) with access to Llama 3.1 8B
- (Recommended) A machine with NVLink or RDMA enabled GPUs



Ask AI

Note

You can use `ucx_perftest` to check the GPU-GPU memory transfer and verify the NVLink or RDMA connection. Please refer to this link: [UCX Performance Test](#).

Architecture Overview

The disaggregated prefill setup consists of three main components:

1. **Prefiller Server (Port 8100)**: Handles the prefill phase of LLM inference
2. **Decoder Server (Port 8200)**: Manages the decoding/generation phase
3. **Proxy Server (Port 9000)**: Coordinates between prefiller and decoder

Configuration

1. Prefiller Server Configuration (`lmcache-prefiller-config.yaml`):

```
local_cpu: False

# PD-related configurations
enable_pd: True
transfer_channel: "nixl" # Using NIXL for transfer
pd_role: "sender"        # Prefiller acts as KV cache sender
pd_proxy_host: "localhost" # Host where proxy server is running
pd_proxy_port: 7500       # Port where proxy server is listening
pd_buffer_size: 1073741824 # 1GB buffer for KV cache transfer
pd_buffer_device: "cuda"  # Use GPU memory for buffer
```

2. Decoder Server Configuration (`lmcache-decoder-config.yaml`):

```
local_cpu: False

# PD-related configurations
enable_pd: True
```



```
transfer_channel: "nixl" # Using NIXL for transfer
pd_role: "receiver"      # Decoder acts as KV cache receiver
pd_peer_host: "localhost" # Host where decoder is listening
pd_peer_init_port: 7300   # Port where initialization happens
pd_peer_alloc_port: 7400  # Port for memory allocation
pd_buffer_size: 1073741824 # 1GB buffer for KV cache transfer
pd_buffer_device: "cuda"  # Use GPU memory for buffer
```

Step-by-Step Setup

1. Environment Setup

Set your Hugging Face token before running the vLLM servers.

```
export HF_TOKEN=your_hugging_face_token
```

2. Launch the vLLM + LMCache Inference Servers

You can launch the components individually:

1. Launch Decoder (on GPU 1):

```
UCX_TLS=cuda_ipc,cuda_copy,tcp \
LMCACHE_CONFIG_FILE=lmcache-decoder-config.yaml \
CUDA_VISIBLE_DEVICES=1 \
vllm serve meta-llama/Llama-3.1-8B-Instruct \
--port 7200 \
--disable-log-requests \
--kv-transfer-config \
'{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_consumer","kv_con
```

2. Launch Prefiller (on GPU 0):

```
UCX_TLS=cuda_ipc,cuda_copy,tcp \
LMCACHE_CONFIG_FILE=lmcache-prefiller-config.yaml \
CUDA_VISIBLE_DEVICES=0 \
vllm serve meta-llama/Llama-3.1-8B-Instruct \
```



```
--port 7100 \  
--disable-log-requests \  
--kv-transfer-config \  
'{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_producer","kv_con
```

3. Launch a proxy server to coordinate between prefiller and decoder:

The code for the proxy server is available in vLLM repo

```
python3 ../disagg_proxy_server.py \  
--host localhost \  
--port 9100 \  
--prefiller-host localhost \  
--prefiller-port 7100 \  
--num-prefillers 1 \  
--decoder-host localhost \  
--decoder-port 7200 \  
--decoder-init-port 7300 \  
--decoder-alloc-port 7400 \  
--proxy-host localhost \  
--proxy-port 7500 \  
--num-decoders 1
```

Note

The `UCX_TLS` environment variable is used to specify the transport layer for UCX (the example uses NVLink) The `CUDA_VISIBLE_DEVICES` environment variable is used to specify the GPUs to use for the servers.

3. Verify Setup

The servers are ready when you can access:

- Prefiller: `http://localhost:7100/v1/completions`
- Decoder: `http://localhost:7200/v1/completions`
- Proxy: `http://localhost:9100/v1/completions`



Usage

Send requests to the proxy server (port 9000) using either the completions or chat completions endpoint:

```
curl http://localhost:9000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-3.1-8B-Instruct",
    "prompt": "Tell me a story",
    "max_tokens": 100
  }'
```

You can also test the setup with the following command, which runs vLLM's serving benchmark:

```
git clone https://github.com/vllm-project/vllm.git
cd vllm/benchmarks
vllm bench serve --port 9000 --seed $(date +%s) \
  --model meta-llama/Llama-3.1-8B-Instruct \
  --dataset-name random --random-input-len 5000 --random-output-len 200 \
  --num-prompts 50 --burstiness 100 --request-rate 1
```

Monitoring

The prefiller instance will log the throughput of KV cache transfer:

```
LMCache INFO: Store 5271 tokens takes: 6.5000 ms, throughput: 98.9889 GB/s;
offload_time: 2.6594 ms, put_time: 3.4539 ms
(cache_engine.py:190:lmcache.v1.cache_engine)
```

The decoder instance will log how many tokens are fetched from the L

 Ask AI

```
LMCache INFO: Reqid: cmpl-b8bf01cbe47e4d108732ceeb4158d310-0, total tokens 5170,
LMCache hit tokens: 5169, need to load: 5169
(vllm_v1_adapter.py:543:lmcache.integration.vllm.vllm_v1_adapter)
```

The proxy server will log the TTFT of the prefiller node:

```
=====
Num requests: 49
Prefill node TTFT stats:
- Average (ms): 0.1530598815606565
- Median (ms): 0.15739011764526367
- 99th Percentile (ms): 0.1643616008758545
=====
```

Troubleshooting

Common issues and solutions:

- 1. **GPU Requirements:** Ensure you have at least 2 GPUs available
- 2. **Port Conflicts:** Check if ports used above are available
- 3. **HF Token:** Verify your token starts with `hf_` and has necessary model access
- 4. **CUDA Errors:** Ensure `CUDA_VISIBLE_DEVICES` is set correctly for each server

< [Example: Share KV cache across multiple LLMs](#)

[Example: Multimodal KV Cache Support](#) >

